Football Analytics Telegram Bot — Technical Blueprint

A research-backed architecture for a Telegram-delivered football analytics system: multi-engine data pipeline, statistical + ML models, calibrated probabilities, and a progressive-disclosure Telegram UX.

---

1. System Architecture Overview

The system is a pipeline, not a monolith. Data flows one direction through independent, swappable stages:

```
Providers → Data Engine → Validation Engine → Normalization Engine → Feature Store
    → [Form | H2H | Team Strength] Engines → [Statistical | ML] Engines
    → Ensemble Engine → Calibration/Eval Engine → AI Analysis Engine → Telegram Bot
```

Each engine is a Python module with one job, one input contract, and one output contract. None of them talk to Telegram directly, and none of them call external APIs directly except the Data Engine — this is what lets you swap a data provider or add a model without touching the bot code.

---

2. Data Providers

Nine providers were compared across coverage, xG availability, pricing, and production-readiness (2026 pricing).

Primary — Sportmonks or API-Football. Both offer strong European league coverage, xG as an add-on/native field, and REST APIs with stable IDs. Sportmonks is stronger on proprietary xG and structured includes; API-Football is cheaper at entry ($19/mo Pro tier) and has a large free tier for prototyping.

Secondary (cross-validation) — TheStatsAPI or football-data.org. Used specifically to catch discrepancies against the primary provider (e.g., conflicting shot counts). TheStatsAPI bundles odds, xG, and shotmaps on one flat plan; football-data.org is cheaper but thinner on advanced stats.

Validation/backfill — StatsBomb Open Data. Free, GitHub-published event-level data. Not a live API — no fixtures or real-time scores — but excellent for offline model training, backtesting, and validating your normalization logic against a known-clean dataset.

Avoid Understat and FBref as production data sources: they have no official API, require scraping, and carry ToS risk for a commercial product — fine for a research notebook, not for something you're shipping to users.

If you later need official/rights-cleared feeds (broadcast-grade, regulatory contexts) or 80+ sport coverage, Sportradar and Genius Sports are the enterprise options, at substantially higher cost.

Design constraint: every provider integration implements the same internal `DataProvider` interface (`get_fixtures()`, `get_team_stats()`, `get_h2h()`, `get_lineups()`, …). Swapping or adding a provider means writing one adapter class, not touching the rest of the app.

---

3. Data Pipeline

Engine 1 — Data Engine
Pulls from all configured providers on a schedule (pre-match: T-48h, T-24h, T-2h for lineups/injuries) and on-demand when a user requests analysis. Writes raw, provider-tagged responses to a staging table before anything is trusted.

Engine 2 — Data Validation Engine
Checks each incoming payload for completeness (required fields present), plausibility (a team can't have negative shots, possession must sum to ~100%), duplicate matches (same fixture from two providers), and staleness (data older than an acceptable window for its type). Flags — doesn't silently drop — anything suspicious.

Engine 3 — Normalization Engine
This is the unglamorous engine that determines whether the whole system is trustworthy. It resolves:

- **Team name mapping** — "Man Utd" / "Manchester United" / "Man United FC" → one canonical ID.
- **Statistic definitions** — providers don't always count "shots on target" or "big chances" the same way; document each provider's definition and normalize to one schema.

- **Conflict resolution** — when Provider A says 12 shots and Provider B says 14, don't average blindly. Apply a **provider priority order** (primary > secondary > tertiary) per stat type, informed by which provider has historically been more consistent for that stat, and record the disagreement itself as a data-quality signal rather than hiding it.

Every match that comes out of this engine carries a **Data Quality Score** (0–100%) derived from: source agreement rate, completeness of expected fields, and data freshness.

---

4. Feature Engineering

Engine 4 — Form Engine
Rolling-window performance (last 5/10 matches) with exponential recency weighting rather than a flat average — a team's form from 8 games ago should matter less than last week's. Separate home-form and away-form windows, since home/away splits are one of the strongest signals in football data.

Engine 5 — H2H Engine
Historical meetings between the two specific

teams, weighted by recency and by whether the venue/competition matches the upcoming fixture. H2H is a weaker signal than form in most research but still adds value, especially for derbies and stylistic mismatches.

Engine 6 — Team Strength Engine
Combines three complementary signals rather than picking one:

- **Elo-style ratings**, updated match-by-match, capture overall trajectory.
- **Attack/defense ratings** (goals-for and goals-against relative to league average, opponent-adjusted) feed directly into the Poisson-family models below.
- **Opponent-adjusted form** — form data means little without knowing the quality of opposition faced.

---

5. Statistical Models

Engine 7 — Statistical Engine

The **Dixon-Coles model** (Dixon & Coles, 1997) is the standard baseline in football analytics research: it extends a simple independent-Poisson

goals model with (a) a low-score correction, since real matches have more draws and tight scorelines than independent Poisson predicts, and (b) exponential time-weighting so recent matches count more than older ones. Academic work since 1997 has extended it further (bivariate/Sarmanov dependence structures, dynamic state-space versions), but the core Dixon-Coles formulation remains a strong, interpretable, cheap-to-compute baseline and is a reasonable first statistical engine to implement.

Output: a full scoreline probability matrix (e.g., P(1-0), P(2-1), …), from which match-outcome probabilities, over/under, and both-teams-to-score probabilities can all be derived — this is more useful than a model that only outputs 1X2 directly.

Engine 8 — Machine Learning Engine

Research consistently finds that gradient-boosted trees (XGBoost/LightGBM) tend to edge out logistic regression and random forests on tabular football data, while logistic regression remains a useful, interpretable baseline and random forests a reasonable middle ground. A recurring, important finding across the literature: **bookmaker odds are a very hard baseline to beat** — treat closing odds

(where available) as both a feature and a benchmark your ensemble should be compared against, not just a competitor to ignore.

Recommended set: Logistic Regression (baseline/interpretability), XGBoost or LightGBM (primary), Random Forest (secondary, for feature-importance sanity checks). Skip deep learning initially — with per-league dataset sizes in the thousands of matches, gradient boosting typically matches or beats neural approaches while being far cheaper to maintain and explain.

Engine 9 — Ensemble Engine

Don't average blindly. Combine the Dixon-Coles output and the ML model outputs using a **stacked/weighted approach**, where weights are learned or tuned based on each model's historical calibration per-league (a model might be well-calibrated for the Premier League and poorly calibrated for a lower-data league — the ensemble should reflect that, not treat all leagues identically).

Engine 10 — Calibration & Evaluation Engine

This is the engine that keeps the system honest. Track, per model and per ensemble, over a rolling window:

- **Log loss** and **Brier score** — how well-calibrated the probabilities are, not just whether the favorite won.
- **Calibration curves** — when the model says "60%," does that outcome actually happen ~60% of the time?
- **Walk-forward backtesting** — train only on data available before each match date, never on future data, and re-evaluate continuously as new results come in. This is the single most important discipline in the whole system; without it, you will silently overfit and won't know until real performance disappoints.

The output feeds directly into the "Model Agreement" and "Data Quality" sections of the Telegram result card, and should also gate the AI Analysis Engine — a match with low model agreement or low data quality should be flagged as such to the user.

Engine 11 — AI Analysis Engine

Runs strictly last, and only on structured outputs from Engines 1–10 — never on raw provider data. Its job is narrow: turn a JSON object of probabilities, feature values, and uncertainty signals into 3–5 sentences of natural-language

explanation ("Arsenal's home xG has outperformed their actual goals by 0.4/game over the last 6 matches, suggesting some finishing under-performance rather than a defensive issue for Chelsea"). Constrain it with a system prompt that explicitly forbids introducing any number not present in the structured input — the AI explains, it doesn't calculate or invent.

---

6. Prediction Philosophy

No "99% accuracy" claims anywhere in the product — that's not how football works, and claiming it undermines credibility with any user who checks results against reality. Realistic, well-calibrated football models land roughly in the 50–55% match-outcome accuracy range for competitive leagues (three-way outcome, where random guessing is ~33%), and even the best models rarely beat closing bookmaker odds by a wide margin. The product's value proposition should be **transparency and calibration**, not a fictional edge: showing users probabilities, which models agree, how confident the system actually is, and what could make it wrong.

Every result should surface:

- Probabilities (not a single "prediction")
- Model agreement across the ensemble
- Data Quality Score
- An explicit uncertainty section naming the biggest risk factors (missing lineup, key injury unconfirmed, small H2H sample, etc.)

---

7. Telegram UX

Navigation flow

```
/start
 └─ ⚽ FOOTBALL ANALYTICS
     ├─ 🔎 Analyze Match → Competition
→ Home Team → Away Team → Run Analysis
     ├─ 📊 Team Analysis
     ├─ 🆚 H2H
     ├─ 📈 Recent Form
     └─ ℹ️ About
```

Use inline keyboards throughout (never force free-text team names — it invites typos and ambiguous matches). Every selection screen needs a **Back** button and, for long team/competition lists, pagination and a search filter.

Loading state

Multi-step loading messages (edited in place via `edit_message_text`, not spammed as new messages) that reflect the real pipeline stages: "Collecting football data…" → "Validating sources…" → "Analyzing recent form…" → "Running statistical models…" → "Comparing model outputs…". This isn't just cosmetic — it sets accurate expectations for a request that may take a few seconds due to live API calls.

Result card — progressive disclosure

The first message is a **summary only**, not a data dump:

```
━━━━━━━━━━━━━━━━━━━━
⚽ FOOTBALL ANALYSIS
Arsenal vs Chelsea
━━━━━━━━━━━━━━━━━━━━
📊 DATA QUALITY: 91%

📈 MODEL PROBABILITIES
Arsenal — XX%   Draw — XX%   Chelsea — XX%

⚽ GOAL PROFILE
Arsenal xG — X.XX   Chelsea xG — X.XX
```

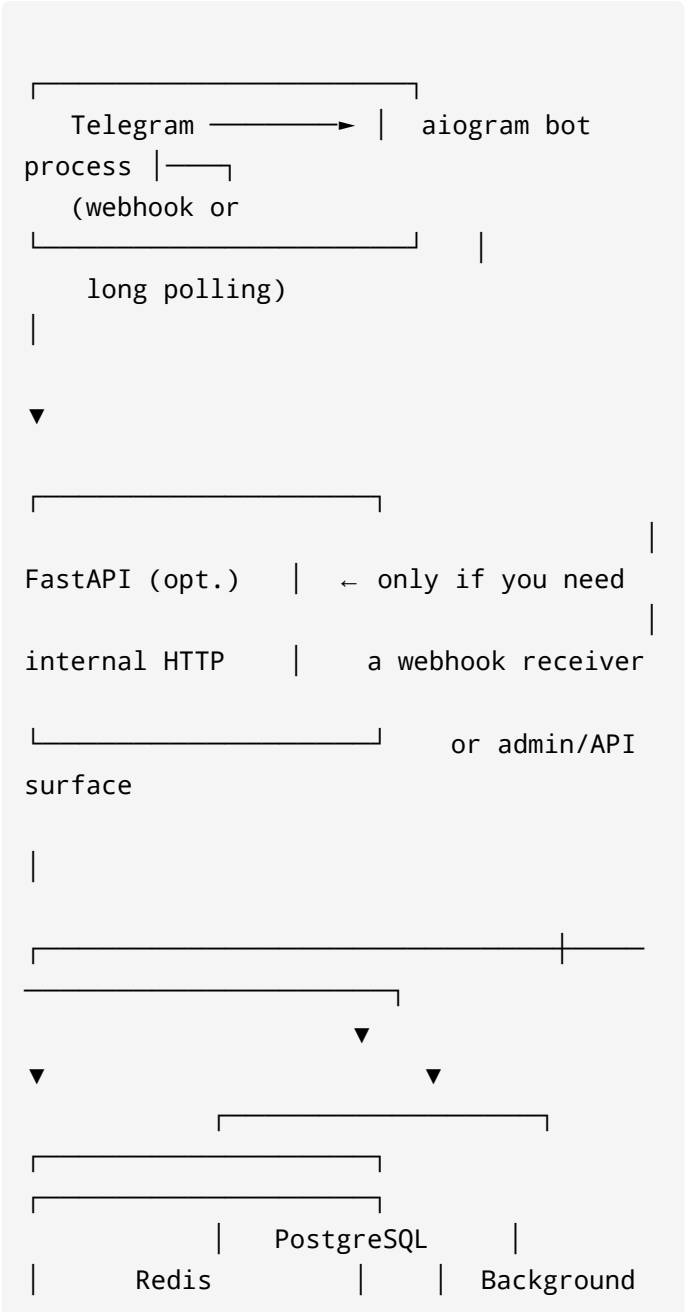
```
🔥 KEY SIGNALS
• Recent form   • Attack strength
• Defensive strength   • H2H   •
Home/away split

⚠️ UNCERTAINTY
[Biggest factors that could make the
model wrong]
──────────────────────
[📊 Full Stats] [📈 Form] [🆚 H2H]
[⚽ Goals]      [🤖 Models] [🧠 AI
Analysis]
```

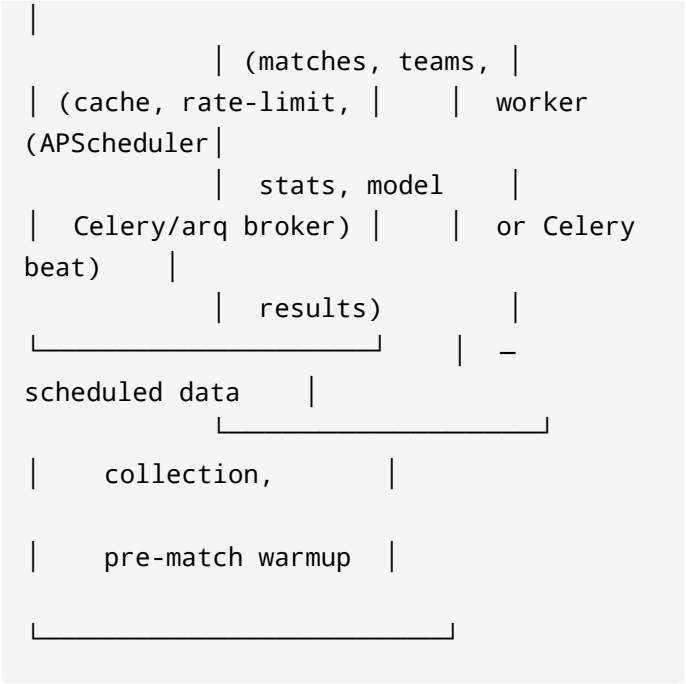
Each button below the card opens a **new, focused message** (or edits into a sub-view with its own Back button) — Full Stats, Form breakdown, H2H history, Goal/xG detail, per-model probability breakdown, and the AI narrative. This keeps the primary card scannable on a phone screen and lets engaged users drill down without overwhelming casual ones.

---

8. VPS Deployment Architecture

Keep it to what's actually needed — this is a single-bot analytics product, not a high-frequency trading system.


Telegram
aiogram bot process
(webhook or long polling)
FastAPI (opt.)
← only if you need internal HTTP
a webhook receiver or admin/API surface
Redis
PostgreSQL
Background



- **aiogram** (async, actively maintained, built-in FSM for multi-step selection flows) over `python-telegram-bot` for this use case — the fully async design fits a bot that's frequently waiting on external API and DB calls.
- **Long polling** is simplest to run and debug on a single VPS; switch to **webhooks behind Nginx + TLS** once you need lower latency or are running multiple bot instances.
- **FastAPI** only if you need webhook ingestion or an internal admin/API surface — don't add it

just because it's common; a polling-only bot doesn't need it.

- **PostgreSQL** for everything durable: normalized match data, model outputs, calibration history, user state.
- **Redis** for response caching (avoid re-hitting provider APIs for a match analyzed twice in an hour), rate-limiting provider calls, and as a broker if you use Celery/arq for background jobs.
- **Background workers** (APScheduler for simple cron-like jobs, or Celery/arq for more complex queued work) handle scheduled data collection and pre-computation, so a user's "Analyze Match" tap isn't blocked on slow provider calls when a fixture is already known in advance.
- **Docker Compose** to run bot + Postgres + Redis + worker as one stack; add Nginx only once you're on webhooks.
- **Logging** to structured JSON (stdout, captured by Docker), **backups** via nightly `pg_dump` to off-VPS storage, **monitoring** via a lightweight uptime check plus periodic log review — skip a full Prometheus/Grafana stack unless usage genuinely justifies it later.

---

9. Python Project Structure

```
app/
├── bot/                    # aiogram
handlers, keyboards, FSM states
│   ├── handlers/
│   ├── keyboards/
│   └── middlewares/
├── providers/               # one
adapter per data source, common interface
│   ├── base.py
│   ├── sportmonks.py
│   ├── api_football.py
│   └── statsbomb.py
├── data/                   # Engine 1
— orchestrates provider calls, staging
├── validation/              # Engine 2
├── normalization/           # Engine 3
— team/name mapping, stat schema
├── analytics/
│   ├── form/                # Engine 4
│   ├── h2h/                 # Engine 5
│   ├── team_strength/       # Engine 6
│   ├── statistics/           # Engine
7 — Dixon-Coles etc.
│   ├── machine_learning/     # Engine
8
│   ├── ensemble/             # Engine
9
│   └── evaluation/           # Engine
10 — calibration, backtesting
```

```
├── ai/                        # Engine
11 — structured-output-to-narrative
├── database/                  #
SQLAlchemy models, migrations (Alembic)
├── workers/                   #
scheduled jobs (APScheduler/Celery tasks)
├── services/                  # cross-
cutting orchestration (e.g. "run full
analysis")
├── schemas/                   # Pydantic
models — the contracts between engines
├── config/
└── tests/
```

One addition worth making explicit: `schemas/` is the load-bearing directory. Every engine's input and output should be a Pydantic model defined there, not a raw dict — this is what actually enforces the "swap one provider/model without rewriting the app" goal, and it makes the AI Analysis Engine's "never invent statistics" constraint mechanically checkable (its input schema simply doesn't contain fields it hasn't been given).

---

10. Recommended Technology Stack

Layer	Choice	Why
Bot framework	aiogram 3.x	Async-native, built-in FSM, actively maintained
Web/API (optional)	FastAPI	Only if webhooks or an admin API are needed
Database	PostgreSQL	Durable structured storage, good JSON support for raw payloads
Cache/broker	Redis	Caching + Celery/arq broker + rate-limiting
Scheduling	APScheduler or Celery beat	Pre-match data collection,

Layer	Choice	Why
		periodic model retraining
Statistical modeling	`penaltyblog` / custom Dixon-Coles implementation	Purpose-built for football; avoids reinventing the Poisson math
ML	XGBoost or LightGBM, scikit-learn (LogReg, RF)	Best tabular performance in the research literature
Validation contracts	Pydantic	Enforces engine input/output schemas
Migrations	Alembic	Standard with SQLAlchemy

Layer	Choice	Why
Deployment	Docker Compose on a Linux VPS	Simplest reliable setup at this scale
Data providers	Sportmonks or API-Football (primary), TheStatsAPI or football-data.org (secondary), StatsBomb Open Data (offline validation/backtesting)	Coverage + cross-validation + free backtesting corpus

---

11. Implementation Roadmap

1. **Data foundation** — provider adapters, staging tables, validation + normalization engines, Data Quality Score. Nothing downstream matters until this is solid.
2. **Statistical baseline** — Dixon-Coles engine end-to-end, backtested with walk-forward evaluation against StatsBomb Open Data. This alone can power a v1 bot.

3. **Telegram bot v1** — navigation flow, loading states, summary result card wired to the Dixon-Coles output only. Ship this before adding ML — it validates the UX and data pipeline under real usage.
4. **ML + ensemble** — add XGBoost/LightGBM, build the ensemble engine, extend calibration tracking to compare ensemble vs. Dixon-Coles-only vs. bookmaker-odds baseline.
5. **AI Analysis Engine** — wire structured outputs into an LLM narrative layer with a strict no-invented-numbers system prompt.
6. **Progressive UX expansion** — Full Stats / Form / H2H / Goals / Models sub-views.
7. **Operational hardening** — scheduled pre-computation for upcoming fixtures, backups, structured logging, basic monitoring.

---

12. Build Blueprint for Coding Assistant

Convert this into implementation phases a coding assistant can execute sequentially:

Phase 1 — Foundations

- Set up `app/` structure above, Docker Compose (bot + Postgres + Redis), Alembic migrations for `teams`, `matches`, `raw_provider_data`.
- Implement `providers/base.py` as an abstract interface (`get_fixtures`, `get_team_stats`, `get_h2h`, `get_lineups`); implement one concrete provider adapter first.
- Implement Engine 2 (validation) and Engine 3 (normalization + team-name mapping table) with unit tests covering conflicting-stat scenarios.

Phase 2 — Statistical core

- Implement Engine 7 (Dixon-Coles) as a standalone, testable module with a clear `fit(matches) -> params` / `predict(home, away) -> score_matrix` interface.
- Implement Engine 10's backtesting harness (walk-forward, log loss, Brier score) and run it against StatsBomb Open Data before touching Telegram at all.

Phase 3 — Bot v1

- Implement `bot/` with aiogram FSM: competition → home team → away team → run analysis.

- Wire the loading-state sequence and the summary result card to Engine 7's output plus the Data Quality Score.

Phase 4 — ML + ensemble

- Add Engine 8 (XGBoost/LightGBM baseline model, feature set drawn from Engines 4–6).
- Add Engine 9 (weighted ensemble) and extend Engine 10 to evaluate ensemble vs. individual models vs. bookmaker-odds baseline where available.

Phase 5 — AI layer + full UX

- Add Engine 11 with a locked-down system prompt (structured JSON in, prose out, no new numbers).
- Add the drill-down sub-views (Full Stats, Form, H2H, Goals, Models, AI Analysis) as separate handlers reusing the same underlying structured result object.

Phase 6 — Operations

- Scheduled pre-fetch/pre-compute for known upcoming fixtures.
- Backup job, structured logging, basic uptime monitoring.

Do not generate full code for all phases at once — implement and test Phase 1 before starting Phase 2, since every later engine depends on the normalization and schema contracts established there.